

University Student-Support Case Agent - Week 4 Tool/Function-Calling Implementation

Group A

Name	Student Number
Namatovu Christine Maria	2300701098
Opiyo Oscar	2300701330
Isio Lily	2300700445
Babirye Sandra	2300707270
Ogwal Richard	2300716574

BSE4104 Emerging Trends in Software Engineering

Prepared against: Week 4 Tool Catalogue and Tool/API Schemas (Isio Lily), Project Charter AI Boundary Matrix (Week 1), Prompt v3.0-RAG. Baseline model: Gemini 2.0 Flash - native function-calling support. Implementation: src/tools.py. Author: Opiyo Oscar.

1. Implementation approach

The four catalogue tools are implemented as plain Python functions in src/tools.py, fronted by a single orchestrator entry point `call_tool(tool_name, args, session)`. The model is never given database or file access: it may only choose a tool name and arguments from `TOOL_SCHEMAS`, and every authorization and validation check runs in deterministic code before the tool body executes. This carries the Week 1 rule that the orchestrator, not the model, is the authority on what a tool call may do.

- `get_timetable` reads `data/timetable.json`; `student_id` must equal session `student_id`; unknown ID returns a structured `not_found` error, never a fabricated schedule.
- `get_case_status` reads `data/cases.json`; the record's stored `student_id` must match the session. Not-found and wrong-owner return identically worded refusals so existence is not leaked.
- `search_knowledge_base` wraps the Week 3 TF-IDF retriever (`src/rag.py`); empty results (`[]`) is the signal the orchestrator uses to force abstention.
- `create_support_case` appends one JSON record to `data/cases.json` with status locked to Pending and created_by agent_draft. The schema has no status field, so a smuggled status is rejected as a schema violation before any record is created.

2. Orchestration layer

`call_tool()` is the only path the application uses to reach a tool. It performs, in order: unknown-tool rejection, schema validation (required fields, patterns, enums, length caps, forbidden privileged keys), dispatch to the tool function, and a malformed-response guard so a tool can never crash the application or return a non-object. Every return carries a tool field naming which tool produced it, so traces stay inspectable.

```
call_tool("get_timetable", {"student_id": "2300701330"}, session)
call_tool("create_support_case", {"student_id": "...", "category": "missing_marks",
    "subject": "...", "description": "..."}, session)
```

3. How the model selects and calls functions

Prompt v3.0-RAG instructs the model to output one JSON tool call per turn (`{"tool": ..., "arguments": {...}}`). The application parses that JSON, passes it to `call_tool()`, and feeds the structured tool response back for the final grounded answer. If the model emits malformed JSON, an unknown tool, or arguments that fail validation, no tool executes and the agent asks a clarifying question instead of guessing. There is no code path by which the model can invent a fifth tool.

- Policy question -> `search_knowledge_base` -> grounded answer with [Source: DOC-xx].
- Timetable request -> `get_timetable` -> verbatim entries from the store.
- Case-status request -> `get_case_status` -> status only for cases the student owns.
- New-case request -> `create_support_case` -> draft CASE-XXXX in Pending (human approval handled in Richard's safety layer).

4. Structured tool responses

Success returns the catalogue output schema plus a tool field. Failure returns a small error envelope the agent maps to user-safe wording:

```
{"error": "invalid_arguments | unauthorized | not_found | service_unavailable"
"message": "human-readable cause", "tool": "<tool-name>", ...}
```

- `invalid_arguments`: schema violation; tool never executes; agent asks for the missing field.
- `unauthorized`: session mismatch; logged as a boundary event; agent refuses.
- `not_found`: unknown ID or unowned case; identically worded to avoid leaking existence.
- `service_unavailable`: missing or malformed store file, or retriever crash; agent asks to retry later.

5. Demonstration evidence

Command: `python src/tools.py --demo (session student 2300701330)`. Observed output:

```
get_timetable      2 entries for 2300701330
get_case_status    CASE-1001 status=Pending
search_knowledge_base 2 results, top=DOC-01
create_support_case draft CASE-1003 status=Pending
```

Single-call form: `python src/tools.py --call get_timetable --args '{"student_id": "2300701330"}'`. The CASE-1003 draft created during the demo run was reverted so `data/cases.json` keeps only the two synthetic seed records. Write-path evidence (draft creation, approval gate, failure cases) is covered in Richard's authorization and human-approval documentation, which builds on this layer.